ЗАПОРІЗЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
Сучасні каркаси web-додатків
Лісняк Андрій Олександрович

Лабораторна робота 5. Використання Vue composable

Мета

Навчитися розподіляти відповідальність між Vue-компонентами й composables, керувати асинхронним пошуком із debounce та скасуванням запитів і дозавантажувати сторінки API під час прокручування каталогу.

Теоретичні відомості

У роботі поєднано чотири механізми: спостереження за реактивними даними, composables, гнучкі вхідні параметри та браузерний `IntersectionObserver`. Вони розв'язують різні задачі. Спостерігач визначає, коли виконати побічний ефект; composable володіє станом і ресурсами цього ефекту; `toValue()` нормалізує його вхід; `IntersectionObserver` повідомляє, коли нижній маркер каталогу наблизився до видимої області.

1. Спостерігачі у Vue

Реактивне значення саме по собі лише зберігає стан. Щоб у відповідь на його зміну виконати дію — скасувати запит, запустити таймер або отримати нову сторінку — використовують спостерігач.

Vue має два основні засоби спостереження: `watch()` і `watchEffect()`.

`watch`: джерело задається явно

`watch()` відокремлює джерело від побічного ефекту. Він викликає callback, коли значення джерела змінилося, і передає нове та попереднє значення.

```
const query = ref('')

watch(query, (nextQuery, previousQuery) => {
  console.log({ nextQuery, previousQuery })
})
```

Джерелом може бути `ref`, гетер, реактивний об'єкт або масив джерел. Для властивості реактивного об'єкта передають гетер, а не поточне значення:

```
watch(() => filters.category, (category) => {
  // category — нове значення filters.category
})
```

У цій роботі `watch()` доречний там, де залежність має бути очевидною:

- сирий `query` змінився — активний HTTP-запит більше не відповідає наміру користувача, тому його потрібно скасувати;

- змінився пошуковий рядок після debounce — потрібно скинути `skip` і накопичені товари;
- надійшла нова сторінка API — її потрібно замінити або додати до каталогу.

Для негайного скасування запиту використовується `{ flush: 'sync' }`. Синхронний callback виконується під час зміни значення, до звичайної черги оновлень Vue. Такий режим придатний для короткої операції `abort()`, але не для важких обчислень або спостереження за масивом із багатьма синхронними змінами.

`watchEffect`: залежності визначаються під час виконання

`watchEffect()` запускає callback одразу й автоматично запам'ятовує всі реактивні значення, прочитані під час його синхронного виконання.

```
watchEffect(() => {
  document.title = 'Знайдено: ' + total.value
})
```

Після зміни `total.value` ефект виконається знову. На відміну від `watch()`, окремий список джерел тут не задається.

Для асинхронного callback діє важливе обмеження: Vue відстежує лише значення, прочитані до першого `await`. Тому URL запиту потрібно отримати на початку ефекту:

```
watchEffect(async (onCleanup) => {
  const currentUrl = toValue(url)
  const controller = new AbortController()

  onCleanup(() => controller.abort())

  const response = await fetch(currentUrl, {
    signal: controller.signal,
  })
})
```

Якщо викликати `toValue(url)` лише після `await`, зміни реактивного URL не стануть залежністю цього ефекту.

Очищення попереднього запуску

Перед повторним запуском спостерігача Vue викликає зареєстровану функцію очищення. Вона має звільнити ресурс, створений саме цим запуском:

- `clearTimeout()` видаляє попередній debounce-таймер;
- `AbortController.abort()` скасовує застарілий запит;
- функція відписки припиняє слухання події або іншого джерела.

Це захищає від змагання запитів. Без скасування повільна відповідь для старого рядка може надійти після нової та перезаписати актуальні дані.

Скасований запит є очікуваним завершенням роботи, тому він не повинен відображатися як помилка інтерфейсу.

Debounce і скасування не замінюють одне одного. Debounce відкладає створення наступного запиту, а `abort()` припиняє запит, який уже виконується. У лабораторній сирий `query` керує негайним скасуванням, а відкладений `debouncedQuery` — формуванням нового URL.

2. Composable як власник реактивної логіки

За визначенням Vue, composable — це функція, яка використовує Composition API для інкапсуляції та повторного використання логіки зі станом. Назва такої функції за домовленістю починається з `use`.

Звичайна функція перетворення отримує аргумент і одразу повертає результат. Composable може зберігати реактивний стан, створювати спостерігачі, реєструвати lifecycle hooks і змінювати результат із часом.

```
export function useRequestState() {
  const data = ref(null)
  const error = ref(null)
  const isLoading = ref(false)

  return { data, error, isLoading }
}
```

Рекомендований контракт повернення — звичайний об’єкт, властивостями якого є `ref`. Його можна деструктурувати без втрати реактивності:

```
const { data, error, isLoading } = useFetch(url)
```

Кожен виклик composable створює окремий стан, якщо цей стан оголошено всередині функції. Два компоненти, які викликали `useFetch()`, не повинні випадково використовувати один `isLoading` або один `AbortController`.

Межа відповідальності

Composable інкапсулює механізм, але не розмітку. Наприклад, `useFetch` знає, як виконати й скасувати запит, однак не вирішує, як виглядає картка товару. Компонент показує дані та передає події, але не повинен відтворювати власну реалізацію мережевого запиту або observer.

У цій роботі межі мають такий зміст:

Модуль	**Ресурс або стан, яким він володіє**
`useFetch`	HTTP-запит, `AbortController`, `data`, `error`, `isLoading`
`useDebouncedValue`	таймер і відкладене реактивне значення
`useIntersectionObserver`	екземпляр `IntersectionObserver` і підписка на ціль
компонент	розмітка, пропси, події та локальний стан представлення

Модуль, який створив ресурс, відповідає і за його очищення. Компонент не має знати ідентифікатор таймера з `useDebouncedValue`, а `App.vue` не повинен викликати `disconnect()` для observer, створеного всередині composable.

Composable викликають синхронно у `<script setup>` або `setup()`. Тоді Vue може прив'язати його спостерігачі та lifecycle hooks до поточного екземпляра компонента й зупинити їх під час розмонтування.

3. Передавання параметрів і `toValue()`

Найпростіший composable приймає звичайне значення. Для повторно використовуваної реактивної логіки цього часто недостатньо: споживач може мати готовий `ref` або потребувати обчислювати параметр із кількох станів.

Vue використовує тип `MaybeRefOrGetter<T>` для трьох припустимих форм входу:

Форма	**Приклад**	**Поведінка**
звичайне значення	`useFetch('/api/products')`	значення не змінюється реактивно
`ref`	`useFetch(url)`	зміна `url.value` повторює ефект
гетер	`useFetch(() => '/api/products?q=' + query.value)`	ефект залежить від значень, прочитаних гетером

Гетер особливо корисний для пропсів і похідних параметрів. Передавання `props.id` копіює його поточне значення, тоді як `() => props.id` зберігає реактивний зв'язок із пропсом.

Нормалізація входу

`toValue()` повертає однаковий результат незалежно від форми аргументу:

```
toValue('/api/products')
toValue(ref('/api/products'))
toValue(() => '/api/products')
```

У всіх трьох випадках результатом буде рядок `'/api/products'`. Для `ref` функція читає `.value`, для гетера — викликає його, а звичайне значення повертає без змін.

`unref()` також розкриває `ref`, але не викликає гетер. Якщо передати до `unref()` функцію, результатом буде сама функція. Саме тому для параметра типу `MaybeRefOrGetter<T>` використовують `toValue()`.

Місце виклику визначає реактивність

Нормалізація має відбутися всередині реактивного ефекту:

```
export function useFetch(url) {
  watchEffect(async (onCleanup) => {
    const currentUrl = toValue(url)
    // currentUrl використовується для поточного запуску
  })
}
```

Якщо виконати `toValue(url)` один раз до `watchEffect()`, composable отримає лише початковий рядок. Спостерігач не побачить читання `url.value` або реактивних залежностей гетера, тому не зможе повторити запит після їхньої зміни.

Гнучкий вхід не означає, що composable має змінювати аргумент. Вхід читається через `toValue()`, а власний стан повертається окремими `ref`. Якщо споживач не повинен змінювати результат напряму, його повертають через `readonly()`.

4. `IntersectionObserver`

`IntersectionObserver` — браузерний API для асинхронного спостереження за перетином елемента з видимою областю або заданим контейнером. Для нескінченного прокручування цільовим елементом стає невеликий маркер після списку товарів.

Observer працює з такими поняттями:

- `target` — DOM-елемент, за яким спостерігають;
- `root` — контейнер спостереження; значення `null` означає viewport;
- `rootMargin` — відступ навколо меж `root`, записаний за правилами CSS `margin`;
- `threshold` — частка видимої площі цілі, після перетину якої викликається callback;
- `IntersectionObserverEntry` — запис зі станом перетину, зокрема `isIntersecting` та `intersectionRatio`.

```
const observer = new IntersectionObserver(
  (entries) => {
    for (const entry of entries) {
      if (entry.isIntersecting) {
        loadMore()
      }
    }
  },
  {
    root: null,
    rootMargin: '280px 0px',
    threshold: 0.01,
  },
)
```

Додатний вертикальний `rootMargin` розширює область спостереження. Маркер вважається близьким до viewport ще до того, як користувач побачить

кінець каталогу. Значення `threshold: 0.01` означає, що callback спрацює після появи приблизно одного відсотка цілі в розширеній області.

На відміну від обробника `scroll`, observer не вимагає на кожній події вимірювати позиції елементів через `getBoundingClientRect()`. Браузер сам відстежує перетин і передає зміни callback. Водночас observer лише повідомляє про подію: він не знає, чи можна зараз завантажувати сторінку.

Перед зміною `skip` застосунок має перевірити:

- попередній запит завершився;
- помилки немає;
- debounce не очікує нового пошукового рядка;
- кількість отриманих товарів менша за `total`.

Один маркер може залишатися в області перетину під час кількох оновлень DOM, тому без цих умов легко надіслати повторні запити з однаковим `skip`.

Життєвий цикл observer

DOM-елемент шаблонного `ref` відсутній до монтування компонента. Тому `IntersectionObserver` створюють у `onMounted()`, після появи цілі викликають `observe(target)`, а під час зміни цілі припиняють спостереження за попереднім елементом.

Під час розмонтування потрібно викликати `disconnect()`. Це від'єднує всі цілі та не дозволяє callback звертатися до стану компонента, якого вже немає. Перевірка наявності `window` та `IntersectionObserver` захищає код у середовищі без DOM, зокрема під час серверного рендерингу.

У підсумку потік нескінченного прокручування має такий вигляд:

```
маркер наблизився до viewport
  → IntersectionObserver повідомив про перетин
  → компонент перевірив умови завантаження
  → skip збільшився
  → реактивний URL змінився
  → useFetch отримав наступну сторінку
  → нові товари додано без повторення id
```

Функціональні вимоги

- Після відкриття сторінки каталог отримує з DummyJSON перші 12 товарів і показує загальну кількість результатів, завантаження, помилку та порожній стан
- Зміна поля пошуку запускає новий запит після 400 мс без введення; URL містить параметр q, а порожній рядок повертає початковий каталог
- Кожна зміна сирого пошукового рядка негайно скасовує активний запит до завершення debounce; скасування не відображається як помилка

- До запитів додано затримку DummyJSON 1500 мс, щоб стан pending і скасування були видимі в панелі Network
- Під час наближення нижнього маркера до видимої області автоматично завантажуються наступні 12 товарів через параметр skip
- Наступна сторінка додається до наявних товарів без повторення id; після отримання всіх результатів нові запити не виконуються
- Новий пошуковий рядок скидає skip, завантажені сторінки й загальну кількість, після чого результат формується з першої сторінки нового запиту
- Помилковий запит можна повторити; під час дозавантаження вже показані картки не зникають
- Сторінка містить видимі ПІБ, групу й відомості про середовище та не має горизонтальної прокрутки за ширин 390 і 1440 пікселів

Технічні вимоги

- Застосунок написано як Vue 3 SFC із JavaScript, Composition API та script setup; Axios, готові бібліотеки запитів, debounce й нескінченного прокручування не використовуються
- App відповідає за композицію сторінки, пошуковий рядок, номер сторінки та об’єднання результатів, але не містить розмітки однієї картки або реалізації браузерних побічних ефектів
- ProductSearchForm підтримує v-model через modelValue та update:modelValue і не виконує API-запитів
- ProductCatalog отримує масив products пропсом, відтворює ProductCard з унікальним key і не змінює стан пошуку або пагінації
- ProductCard отримує один об’єкт product пропсом і відповідає лише за представлення товару
- CatalogLoadMore показує стан наступної сторінки, спостерігає власний нижній маркер і надсилає події load-more та retry без зміни списку товарів
- useFetch приймає URL як звичайне значення, ref або гетер, читає його через toValue всередині watchEffect і повертає data, error, isLoading, abort та refetch
- Кожен запуск useFetch має власний AbortController; очищення watchEffect і розмонтування скасовують запит, а застарілий запуск не змінює стан нового
- useDebouncedValue очищує попередній таймер під час зміни джерела та повертає readonly ref
- useIntersectionObserver створює браузерний observer після монтування, повторно підписується після зміни target і викликає disconnect під час розмонтування
- URL формується через URLSearchParams із параметрами limit, skip, delay та, за наявності пошуку, q; сторінки об’єднуються без мутації відповіді API

- Стилі збережено у BEM-класах стартера; готовий дизайн не замінено бібліотекою компонентів або набором utility-класів у шаблоні

Необхідне обладнання та матеріали

- Node.js 24 LTS, npm, Git і редактор коду
- Заготовка каталогу з репозиторію znu-lab-starters
- Браузер із DevTools, панеллю Network і емуляцією розмірів екрана
- Доступ до публічного API DummyJSON

Хід роботи

1. Отримання та перевірка стартера

У корені репозиторію `mjsf-mag` отримайте відкритий репозиторій стартерів і скопіюйте заготовку до каталогу `lab5`:

```
git clone --depth 1 https://github.com/lisnayk/znu-lab-starters.git
../znu-lab-starters
cp -R ../znu-lab-starters/modern-frameworks/labs/05-
composables/starter ./lab5
cd lab5
npm ci
npm run dev
```

Заповніть у підвалі сторінки власні ПІБ, групу й відомості про середовище. Після відкриття сторінки зачекайте на штучну затримку API. Каталог має показати 12 товарів із DummyJSON.

Перевірте дві навмисно незавершені можливості. Введення `phone` не змінює список і не створює нового запиту, а прокручування до нижнього блока не завантажує наступну сторінку. Зафіксуйте початковий стан у таблиці перевірок.

2. План розподілу відповідальності

Перед зміною логіки створіть у README таблицю з колонками «Модуль», «Входи», «Виходи», «Власний стан» і «Відповідальність». Заповніть її для таких модулів:

Модуль	**Межа відповідальності**
`App.vue`	Поєднання стану пошуку, сторінок, composables і компонентів сторінки
`ProductSearchForm.vue`	Поле пошуку, доступний підпис і стан запиту, але не HTTP
`ProductCatalog.vue`	Список отриманих товарів і вибір між сіткою та порожнім станом
`ProductCard.vue`	Представлення одного товару

Модуль	Межа відповідальності
`CatalogLoadMore.vue`	Нижній маркер, стан наступної сторінки та події `load-more` і `retry`
`useFetch.js`	Запит, асинхронний стан і скасування
`useDebouncedValue.js`	Відкладене реактивне значення й очищення таймера
`useIntersectionObserver.js`	Підписка на перетин елемента й очищення observer

Компонент не отримує дані, яких не потребує його шаблон або рішення в межах власної відповідальності. Composable не повертає розмітку й не знає назви компонента, який його викликає.

Створіть структуру:

```
src/
├── api/
│   └── products.js
├── components/
│   ├── CatalogLoadMore.vue
│   ├── ProductCard.vue
│   ├── ProductCatalog.vue
│   └── ProductSearchForm.vue
├── composables/
│   ├── useDebouncedValue.js
│   ├── useFetch.js
│   └── useIntersectionObserver.js
├── styles/
│   └── main.css
├── App.vue
└── main.js
```

3. Перенесення розмітки до компонентів

Спочатку розподіліть готову розмітку без зміни поведінки. Після кожного перенесення перевіряйте сторінку у браузері, щоб знайти втрачений клас або пропс у межах одного кроку.

`ProductCard` отримує обов'язковий пропс `product`. Перенесіть до нього розмітку `article.product-card` і форматування ціни. Компонент не читає API, не знає позицію картки в масиві й не змінює товар.

`ProductCatalog` отримує пропси `products`, `isLoading` та `error`. Він відтворює `ProductCard` через `v-for`, використовує `product.id` як `key` і показує готові стани початкового завантаження, помилки та порожнього результату. Повторний запит передайте подією `retry`.

`ProductSearchForm` реалізує контракт `v-model`. Оголосіть пропс `modelValue` типу `String` і подію `update:modelValue`. Поле передає введений рядок назовні, але компонент не створює `fetch`, URL або таймер.

`CatalogLoadMore` отримує `loaded`, `total`, `isLoading`, `error` і `hasMore`. Він показує готовий нижній блок і надсилає події `load-more` та `retry`.

Спостереження за маркером буде додано після створення відповідного composable.

Після рефакторингу початкові 12 товарів мають виглядати так само, як у стартері. У `App.vue` не повинно залишитися розмітки `product-card`.

4. Composable для HTTP-запиту

Перенесіть `fetch` і пов'язані стани зі стартера до `useFetch.js`. Функція приймає URL у формі звичайного значення, `ref` або гетера. Усередині `watchEffect()` прочитайте його через `toValue()` до першого `await`.

Кожен запуск ефекту створює власний `AbortController`. Передайте його `signal` до `fetch` і зареєструйте `controller.abort()` через `onCleanup`. Під час нового запуску попередній контролер також потрібно скасувати.

Composable повертає такий публічний контракт:

```
return { data, error, isLoading, abort, refetch }
```

Метод `abort()` скасовує активний запит і завершує стан завантаження. Метод `refetch()` змінює внутрішній реактивний лічильник, прочитаний ефектом, тому повторює запит без зміни URL.

У `catch` не записуйте помилку після скасування сигналу. У `finally` змінюйте `isLoading` лише тоді, коли завершився поточний контролер. Старий запуск не повинен вимкнути індикатор нового запиту.

Підключіть `useFetch` у `App.vue` до початкового URL і переконайтеся, що завантаження, помилка та повтор запиту працюють після видалення початкового `fetch` із компонента.

5. Реактивний URL і debounce

Змініть `createInitialProductsUrl()` у `src/api/products.js` на `createProductsUrl(query, skip = 0)`. Побудуйте параметри через `URLSearchParams`. Кожен URL містить `limit=12`, поточний `skip` і `delay=1500`. Для непорожнього рядка використовуйте шлях `/products/search` і параметр `q`; для порожнього — `/products` без `q`.

Створіть `useDebouncedValue(source, delay = 400)`. Початкове значення одержіть через `toValue(source)`. Спостерігайте за нормалізованим джерелом, створюйте таймер для нового значення й видаляйте його через функцію очищення спостерігача. Поверніть `readonly`-посилання, щоб споживач не міг обійти затримку прямим присвоєнням.

У `App.vue` поєднайте сирий і відкладений стани:

```
const query = ref('')
const debouncedQuery = useDebouncedValue(query, 400)
const requestUrl = computed(() =>
  createProductsUrl(debouncedQuery.value, skip.value),
)
```

Передайте `query` у `ProductSearchForm` через `v-model`. Реактивний URL передайте до `useFetch`.

Додайте окремий `watch()` для сирого `query`, який викликає `abort()` із параметром `{ flush: 'sync' }`. Скасування має відбутися під час зміни поля, а не через 400 мс разом із новим URL.

У Network уведіть `phone` кількома короткими порціями. Активний запит має отримати стан `canceled`, а запит з остаточним `q=phone` має початися після паузи. Скасування не повинно відкривати блок помилки.

6. Накопичення сторінок

Створіть в `App.vue` реактивні значення `skip`, `products` і `total`. Спостерігайте за `data`, яку повертає `useFetch`. Відповідь із `skip=0` замінює список. Для наступної сторінки додайте лише товари, id яких ще немає в `products`.

Обчисліть `hasMore` як порівняння кількості завантажених товарів із `total`. Функція `loadMore()` збільшує `skip` на 12 лише за таких умов:

- ще існують результати;
- запит не виконується;
- debounce не очікує нового рядка;
- помилки немає.

Після зміни `debouncedQuery` синхронно поверніть `skip`, `products` і `total` до початкового стану. Новий пошук не повинен додавати свої результати до попередньої видачі.

7. Composable для нижнього маркера

Створіть `useIntersectionObserver(target, callback, options = {})`. Браузерний `IntersectionObserver` ініціалізуйте в `onMounted()`. Якщо API відсутній у `window`, завершіть монтування без помилки.

Після появи DOM-елемента викликайте `observer.observe(element)`. Після зміни цільового елемента припиняйте спостереження за попереднім. Для записів із `entry.isIntersecting === true` викликайте переданий callback. Під час розмонтування зупиніть внутрішній `watch` і викличте `observer.disconnect()`.

У `CatalogLoadMore` створіть шаблонне посилання на кореневий маркер і передайте його до composable. Callback надсилає подію `load-more`. У `App.vue` обробіть її функцією з попереднього етапу.

Для observer використайте такі параметри:

```
{
  rootMargin: '280px 0px',
  threshold: 0.01,
}
```

Додатний `rootMargin` запускає запит до того, як користувач побачить кінець поточної сторінки. Під час запиту збережіть уже завантажені картки й покажіть індикатор лише в нижньому блоці.

8. Перевірка результату

Відкрийте DevTools, увімкніть збереження журналу Network і послідовно виконайте перевірки:

1. Початковий URL містить `limit=12`, `skip=0` і `delay=1500`; після відповіді видно 12 карток.
2. Швидке введення `p`, `ph`, `phone` скасовує активний запит; після 400 мс виконується один актуальний запит із `q=phone`.
3. Пошук рядка, якого немає в назвах, показує порожній стан без помилки в консолі.
4. Очищення поля повертає початковий каталог і запит без параметра `q`.
5. Прокручування додає сторінки з `skip=12` і `skip=24`; попередні картки не зникають, а id не повторюються.
6. Новий пошук після дозавантаження починається з `skip=0` і не залишає товари попередньої видачі.
7. Після отримання всіх знайдених товарів нижній блок повідомляє про завершення й не надсилає нових запитів.
8. Режим Offline показує помилку; після відновлення мережі кнопка повтору завантажує ту саму сторінку.
9. За ширин 390 і 1440 пікселів сторінка не має горизонтальної прокрутки, обрізаного тексту або недоступних кнопок.

Занесіть до таблиці очікуваний результат, фактичний URL, кількість карток і стан інтерфейсу. Збережіть знімки початкового каталогу, пошуку, дозавантаження та Network із canceled-запитом.

Зупиніть dev-сервер клавішами `Ctrl+C` і виконайте виробничу збірку:

```
npm run build
```

9. Завершення роботи

Доповніть README схемою потоку стану:

```
query → abort активного запиту
  └── debounce 400 мс → requestUrl → useFetch → page
                                          └── merge → products
нижній маркер → load-more → skip + 12 ──────────┘
```

Поясніть, чому `ProductSearchForm` не формує URL, `ProductCatalog` не завантажує дані, а очищення таймера, запиту й observer розміщене у відповідних composables.

Закомітьте роботу й надішліть її до репозиторію:

```
git add lab5
git commit -m "Лабораторна робота 5: composables каталогу"
git push
```

Що здається

Здається посилання на репозиторій `mjsf-mag` із проєктом у каталозі `lab5`, README з контрактами модулів, таблиця перевірок, результат виробничої збірки й чотири знімки, перелічені в етапі 8.

Звіт містить титульний аркуш, виконані команди, посилання на репозиторій і видимі дані автора. Звіт завантажується до СЕЗН Moodle у форматі PDF.

Вимоги до змісту та оформлення звіту

- Файл звіту завантажується до СЕЗН Moodle у форматі PDF.
- Звіт містить титульний аркуш і деталі виконання роботи.
- Звіт містить посилання на репозиторій GitHub, Bitbucket або GitLab із роботою в каталозі `lab1`, `lab2`, … за номером роботи.
- Звіт містить команди, які виконуються під час роботи.
- Звіт містить знімки екрана, з яких зрозумілий контекст роботи та видно персональні дані автора: адресний рядок, дані студента й відомості про середовище.

Крім переліченого, за цією роботою здається:

- Вихідний код Vue-проєкту в каталозі lab5 із компонентами ProductSearchForm, ProductCatalog, ProductCard і CatalogLoadMore та composables useFetch, useDebouncedValue і useIntersectionObserver
- README зі структурою компонентів, таблицею їхніх входів, виходів і відповідальностей та описом потоку query → debounce → URL → fetch → products
- Таблиця перевірок із фактичними URL, кількістю карток і станами для початкового завантаження, пошуку, скасування, двох наступних сторінок, порожнього результату та помилки
- Знімок початкового каталогу, знімок відфільтрованої видачі, знімок після дозавантаження і знімок Network із canceled-запитом та наступним успішним запитом
- Результат npm run build і пояснення, чому debounce не замінює abort та чому очищення ресурсу належить composable

Контрольні питання

1. Яке значення має керувати негайним скасуванням активного пошукового запиту?
2. Чому debounce не замінює AbortController під час пошуку через API?
3. Які ресурси мають власне очищення в composables цієї роботи?

4. Який skip відповідає третій сторінці за limit=12?

5. Чому toValue(url) у useFetch потрібно викликати всередині watchEffect до першого await?

Література до роботи

1. Vue.js. Composables — https://vuejs.org/guide/reusability/composables
2. Vue.js. Watchers and side effect cleanup — https://vuejs.org/guide/essentials/watchers.html#side-effect-cleanup
3. Vue.js. Reactivity API: toValue — https://vuejs.org/api/reactivity-utilities.html#tovalue
4. Vue.js. Lifecycle hooks — https://vuejs.org/api/composition-api-lifecycle
5. MDN Web Docs. AbortController — https://developer.mozilla.org/en-US/docs/Web/API/AbortController
6. MDN Web Docs. Intersection Observer API — https://developer.mozilla.org/en-US/docs/Web/API/Intersection_Observer_API
7. MDN Web Docs. IntersectionObserver: rootMargin — https://developer.mozilla.org/en-US/docs/Web/API/IntersectionObserver/rootMargin
8. DummyJSON. Products — https://dummyjson.com/docs/products
9. DummyJSON. Delay responses — https://dummyjson.com/docs#delay-responses